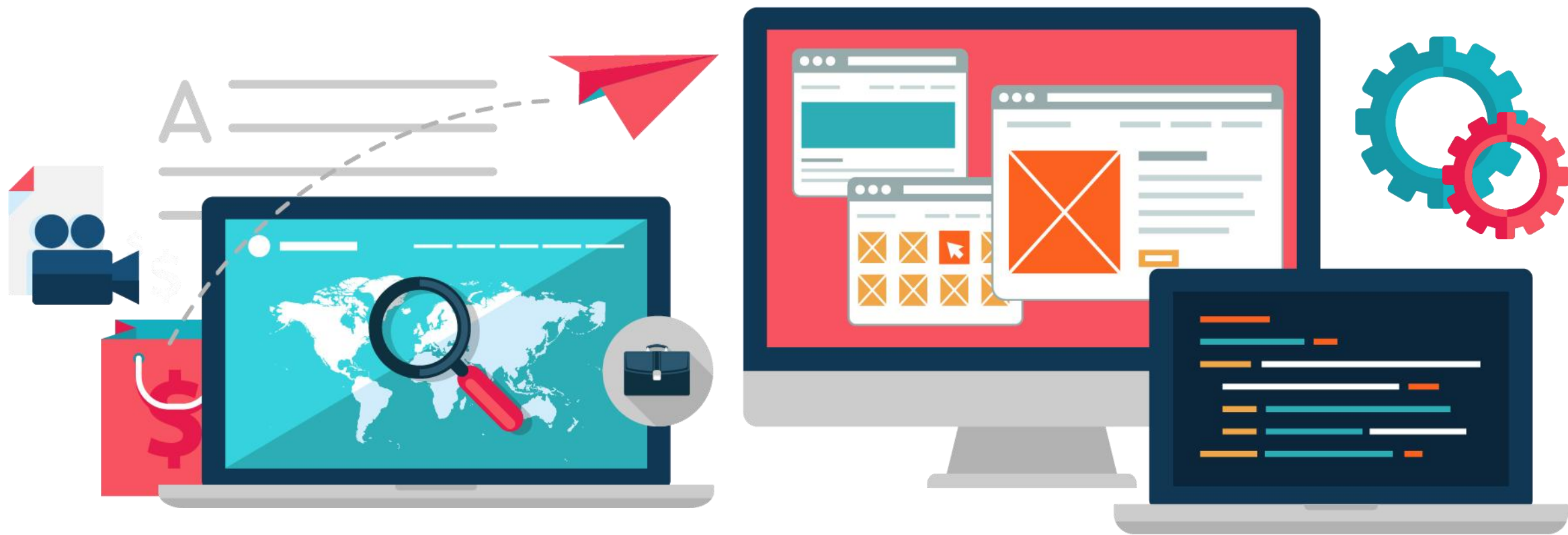


面向对象程序设计

Object Oriented Programming





继承



01

黑盒复用和白盒复用

02

继承和派生

03

派生类中的成员及访问控制

04

派生类的构造和析构函数

05

派生类的拷贝构造和赋值函数

06

newdefine、redefine、overload、overwrite

07

继承的含义

08

继承和组合的选择





- ◆ 拷贝、粘贴、修改
- ◆ 水平关系-普通关联、聚合、组合和依赖---黑盒复用
- ◆ 垂直关系-继承--白盒复用



- ◆ 水平关系-普通关联、聚合、组合和依赖---黑盒复用
- ◆ 黑盒复用是一种功能复用
- ◆ 改变被复用类 (A) 的具体实现, 不会影响复用类 (B、C) 的实现
- ◆ 但要求被复用类具有良好的设计(行为的设计合理、独立)

```
class A {  
public:  
    void AFunc1( );  
    void AFunc2( );  
private:  
    void AFunc3( );  
private:  
    int data;  
};
```



```
//以依赖的方式复用类A的功能  
class B {  
public:  
    void BFunc1( A & a) {  
        BFunc2( );  
        a.AFunc1( );  
    }  
    void BFunc2( );  
private:  
    void BFunc3( );  
};
```

//以关联(普通、聚合、组合)
方式复用类A的功能

```
class C {  
public:  
    void CFunc1( ) {  
        CFunc3 ( );  
        pA->AFunc1( );  
    }  
    void CFunc2( );  
private:  
    void CFunc3( );  
private:  
    A * pA;  
};
```



- ◆ 白盒复用---垂直关系-继承
- ◆ 白盒复用是一种实现复用或代码复用

```
class A {  
public:  
    void AFunc1( );  
    void AFunc2( );  
private:  
    void AFunc3( );  
private:  
    int data;  
};
```

被
复用

//以**关联(普通、聚合、组合)**
方式复用类A的功能(黑盒复用)

```
class C {  
public:  
    void CFunc1( ) {  
        CFunc3 ( );  
        pA->AFunc1( );  
    }  
    void CFunc2( );  
private:  
    void CFunc3( );  
private:  
    A * pA;  
};
```

//以**继承方式**复用类A的实现

```
#include "a.h"  
class C : public A {  
public:  
    void CFunc1( ) {  
        CFunc3 ( );  
        AFunc1( );  
    }  
    void CFunc2( );  
private:  
    void CFunc3( );  
};
```



◆ 继承语法格式

```
class <派生类名> : <继承方式> <基类名称>, ...  
{  
    ....  
};
```

↑
继承列表

◆ 例:

```
class Child : public Parent1, private Parent2  
{  
public:    Child( int n);  
          void ChildFunc( );  
private: int data1;  
          OtherClass * ptr;  
};
```



- ◆ 继承语法格式
class <派生类名> :<继承方式><基类名称>, ...
{ };
- ◆ 继承方式:
public、protected、private三种,
默认为private继承(无明确指定的继承方式时)
- ◆ 基类
 - ◆ 单继承: 只有一个基类
 - ◆ 多(重)继承: 有两个或两个以上的基类
- ◆ 派生类
 - ◆ 派生类可以是其它继承的基类, 即派生类可继续派生
 - ◆ 多层继承和继承树
- ◆ 父类、子类
 - ◆ 在public继承下, 称基类为父类, 派生类为子类



◆ 派生类中的成员(例见下页)

- 派生类的构造、析构、拷贝、赋值函数
- 派生类中定义的成员函数、数据成员
- 基类中的所有成员
(除基类的构造、析构、拷贝、赋值函数)

◆ 派生类中成员的访问控制

◆ 派生类对象的大小



```
class A {  
public:  
    void AFunc1( );  
    void AFunc2( );  
private:  
    void AFunc3( );  
private:  
    int data;  
};
```

//以继承方式复用类A的实现

```
class B : public A {  
public:  
    void BFunc1( ) {  
        BFunc3 ();  
        AFunc1();  
    }  
    void BFunc2( );  
private:  
    void BFunc3( );  
private:  
    int num;  
};
```

派生类B中的成员

- B中缺省的:
构造、析构、拷贝、赋值函数
- B中定义的:
B::BFunc1()
B::BFunc2()
B::BFunc3()
B::num
- 从A中继承的:
A::AFunc1()
A::AFunc2()
A::AFunc3()
A::data

```
class base
{
public:
    base() { m=1; }
    int m;
    static int n;
};
int base::n=0;
class derived: public base
{
public:
    derived() { m=2; }
    int m;
};
```

```
int main()
{
    derived d;
    cout<<sizeof(d)<<endl;
    cout<<d.m<<endl;
    cout<<d.base::m<<endl;
    cout<<base::n<<endl;
    cout<<derived::n<<endl;
    cout<<d.n<<endl;
    return 0;
}
```

对象d中既包括派生类新定义的m，还包括从基类继承过来的m

基类中的静态成员在整个继承层次中只有一个实例，通过基类或派生类都可以访问



- ◆ 派生类中定义的成员----根据指明的访问控制
- ◆ 继承自基类中的成员----见下表

		成员在基类中的访问控制		
		public	protected	private
继承方式	public	public	protected	/
	protected	protected	protected	/
	private	private	private	/

问题：既然基类中private的成员在派生类中都是不可访问的，为什么派生类还要含有这些不可访问的基类数据成员和成员函数？



派生类中成员的访问控制(例)

```
class A {  
public:  
    void AFunc1( ){  
        int i = data;  
    };  
    void AFunc2( );  
private:  
    void AFunc3( );  
private:  
    int data;  
};
```

//以继承方式复用类A的实现

```
class B : public A {  
public:  
    void BFunc1( ) {  
        BFunc3 ();  
        AFunc1();  
    }  
    void BFunc2( );  
private:  
    void BFunc3( );  
private:  
    int num;  
};
```

派生类B中的成员包括:

- B中缺省的:
构造、析构、拷贝、赋值函数 public
- B中定义的:
B::BFunc1() public
B::BFunc2() public
B::BFunc3() private
B::num private
- 从A中继承的:
A::AFunc1() public
A::AFunc2() public
A::AFunc3() 不可见
A::data 不可见

```
class Base
{
public:
    Base( );
    void FuncPub ( );
protected:
    void FuncProt( );
    int num2;
private:
    void FuncPriv( );
    int num1;
};
```

```
class Derived : public Base
{
public:
    Derived( );
    void G1 ( );
protected:
    void G2( );
    int val2;
private:
    void G3( );
    int val1;
};
```

```
Derived::Derived( )
{
    FuncPub( ); //合法
    FuncProt( ); //合法
    FuncPriv( ); //非法
    num2 = 5; //合法
    num1 = 6; //非法
    G1(); //合法
    G2(); //合法
    G3(); //合法
    val2 = 10; //合法
    val1 = 20; //合法
}
```

```
class Base
{
public:
    Base( );
    void FuncPub ( );
protected:
    void FuncProt( );
    int num2;
private:
    void FuncPriv( );
    int num1;
};
```

```
class Derived : public Base
{
public:
    Derived( );
    void G1 ( );
protected:
    void G2( );
    int val2;
private:
    void G3( );
    int val1;
};
```

```
int main() {
    Derived d;
    d.FuncPub( ); //合法
    d.G1();       //合法

    d.FuncProt( ); //protected
    d.num2 = 5;    //protected
    d.G2( );       //protected
    d.val2 = 10;   //protected

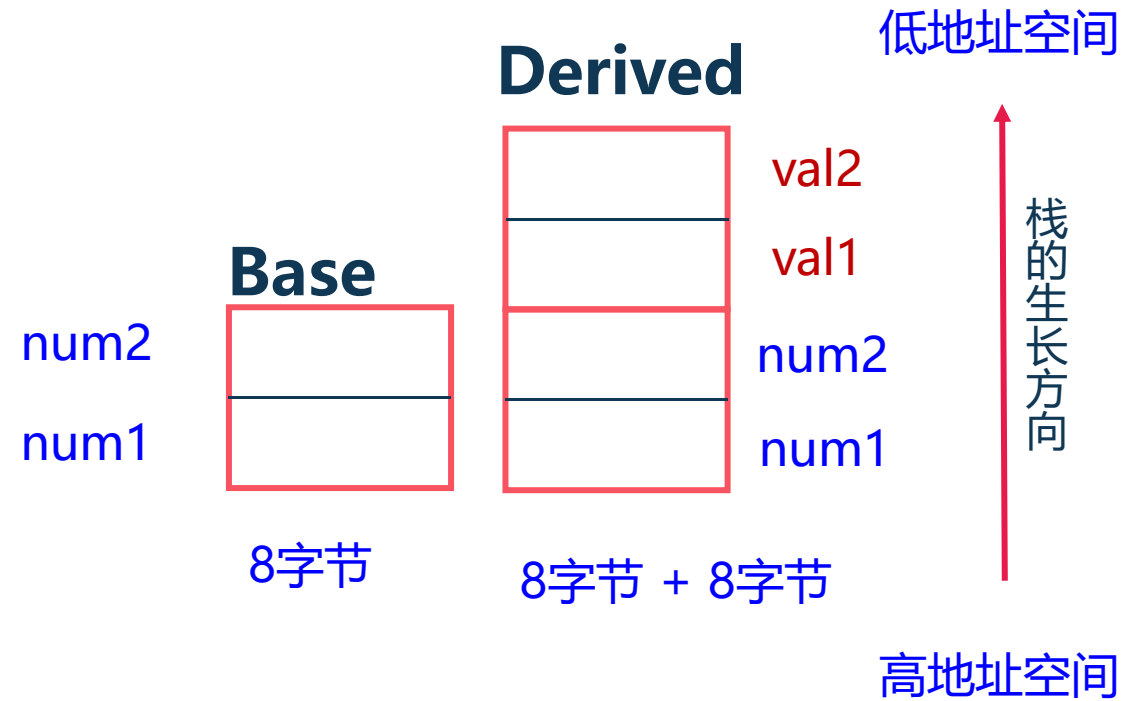
    d.G3( );       //private
    d.val1 = 20;   //private

    d.FuncPriv( ); //不可见
    d.num1 = 6;    //不可见
    return 0;
}
```



```
class Base
{
public:
    Base( );
    void FuncPub ( );
protected:
    void FuncProt( );
    int num1;
private:
    void FuncPriv( );
    int num2;
};
```

```
class Derived : public Base
{
public:
    Derived( );
    void G1 ( );
protected:
    void G2( );
    int val1;
private:
    void G3( );
    int val2;
};
```



特殊情况：基类中没有非静态数据成员，派生类对象的大小？



- ◆ 基类的构造、析构、拷贝、赋值不会被派生类**自动**继承
- ◆ 派生类的构造函数：
 - 构造顺序：先基类，再派生类
 - 初始化列表中可以指定基类的构造函数或拷贝构造函数
 - 基本原则：基类子对象交由基类构造函数初始化，派生类新增成员交由派生类构造函数初始化，分清职责
 - 多重继承时，基类按继承声明的先后顺序构造
- ◆ 派生类的析构函数：
 - 先执行派生类的析构，再自动执行基类析构
- ◆ 派生类不继承这些函数，但编译器隐式生成的版本会自动调用基类的**对应操作**。若基类的特殊成员函数不可用(如private或未定义)，派生类的隐式生成的版本会被删除。



- ◆ 对从基类继承过来的成员的初始化，派生类构造函数最好不要采用函数体内直接赋值的方式，而是通过初始化列表调用直接基类的适当的构造函数来间接初始化，绝不允许在初始化列表对其直接初始化或调用基类其他成员函数初始化
- ◆ 当基类中没有显式定义构造函数时，派生类可以不必显式定义构造函数，或者显式定义构造函数，但初始化列表可以省略对基类构造函数的调用
- ◆ 当基类中显式定义构造函数时，派生类也应该显式定义构造函数，并通过初始化列表调用直接基类的适当的构造函数来初始化从基类继承过来的成员

```
class Base {  
public:  
    Base( int n ) : num(n) {  
        ++num;  
        FuncPriv( );  
    }  
    ~Base( ) {  
        --num;  
        FuncPriv( );  
    }  
private:  
    void FuncPriv( )  
        { cout<<num; }  
    int    num;  
};
```

```
class Derived: public Base {  
public:  
    Derived( int n1,int n2 )  
        :Base(n1),val(n2)  
    { }  
    ~Derived( ) { cout<<val; }  
private: int    val;  
};
```

```
int main( ) {  
    Derived d(1,99);  
    return 0;  
}  
//输出:  
2  99  1
```

```
class A {  
public:  
    A(int num):data(num) { }  
    ~A() { }  
private:  
    int data;  
};
```

```
class A {  
public:  
    A(int num):data(new int(num)) { }  
    ~A() { delete data; }  
private:  
    int * data;  
};
```

```
class B:public A {  
public:  
    B(int n1,int n2):A(n1), num(n2) { }  
    ~B( ) { }  
  
private:  
    int num;  
};
```

派生类的拷贝构造和赋值函数(例)

```
class A {
public:
    A(int num):data(num) { }
    ~A() { }
    A(const A& other) : data(other.data)
    {
    }
    A& operator=(const A& rhs){
        if(this!=&rhs)
            data=rhs.data;
        return *this;
    }
    int GetData() const { return data; }
    operator float( ) { return 0.6*data; }
protected:
    int data;
};
```

```
class B:public A {
public:
    B(int n1,int n2):A(n1), num(n2) { }
    ~B( ) { }
    B(const B& other):A(other)// 若不写:A(other),会自动调用A()
    {num = other. num; }
    B& operator=(const B& rhs) {
        if( this!=&rhs) {
            A::operator=(rhs);//必须写
            num = rhs. num;
        }
        return *this;
    }
private:
    int num;
};
```

注意：应根据是否有指针成员且是否申请了资源，决定是否遵循三五法则，此示例类A和类B实际上不需要显式定义析构、拷贝构造和拷贝赋值。

派生类的拷贝构造和赋值函数(例)

```
class A {
public:
    A(int num):data(new int(num)) { }
    ~A() { delete data; }
    A(const A& other)
        :data(new int(*other.data))
    { }
    A& operator=(const A& rhs){
        if(this!=&rhs){
            int* temp = new int(*rhs.data);
            delete data;
            data = temp;
        }
        return *this;
    }
    int GetData() const { return *data; }
    operator float( ) { return 0.6* (*data); }
private:
    int *data;
};
```

```
class B:public A {
public:
    B(int n1,int n2):A(n1), num(n2) { }
    ~B( ) { }
    B(const B& other) :A(other)// 若不写:A(other),会自动调用A()
        { num = other. num; }
    B& operator=(const B& rhs) {
        if( this!=&rhs) {
            A::operator=(rhs);//必须写
            num = rhs. num;
        }
        return *this;
    }
private:
    int num;
};
```

注意：类A有指针成员且申请了资源，需遵循三五法则，派生类B无指针成员，不需遵循三五法则。明确边界分清职责，谁需要谁显式定义。

◆ 派生类中的成员函数

- 派生类中的构造、析构、拷贝、赋值
若无明确定义，则使用编译器生成的
- 派生类中的静态函数
派生类定义的+继承自基类的
- 继承的基类函数
- 派生类中明确定义的函数

- newdefine: 派生类中新定义的函数(基类中无同名的函数)
- redefine: 派生类中新定义的函数(基类中有同名、同原型的函数)
- overload: 派生类中多个同名的重载函数
- overwrite: 派生类中定义了某个函数，且基类中有同名的函数，则派生类的函数会将基类的同名函数隐藏 (hide)掉。
 - 使用using声明
- override: 若基类中的某个函数是虚的(带virtual关键字)，并在派生类中定义了相同原型的函数，称override

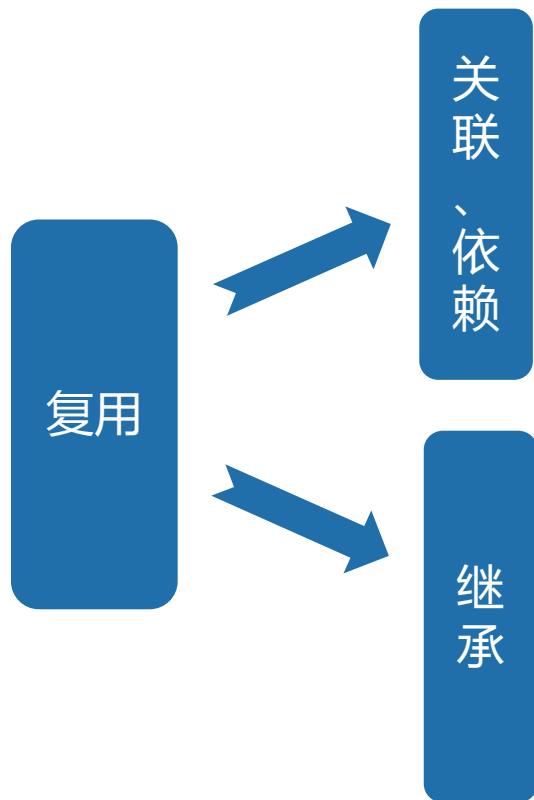


newdefine、redefine、overload、overwrite(hide)(例)

```
class A
{
public:
    void F( );
    void F(int n);
    void G( );
    virtual void H( );
private:
    void K( );
    int data;
};
```

```
class B:public A {
public:
    void BF( ) { } // newdefine的函数
    void G( ) { } // redefine 了基类的G()

    void F(int n,int m) { //或void F(int n)
        // 定义了与基类同名的函数F,(参数不限)
        // 则overwrite了基类的函数F
        // 即新定义的F，隐藏(Hide)了基类中
        // 重载(overload)的F函数.所以
        F( ); //非法
        F(n); //非法
        //可以类定义中F(int,int)前使用using A::F; 语句解决此问题
    }
    virtual void H( )
    { /* override 了基类的虚函数H( ) */ }
};
```



- ◆ 继承是一种代码复用的实现方式
- ◆ 选择继承？ 还是选择关联/依赖？

- ? 功能复用好？ 代码复用好？
- ? 继承有什么好处？
- ? 继承表示的逻辑关系是什么？

- ◆不同的继承方式，表示不同的逻辑含义。
- ◆继承方式：class Derived: <继承方式> Base { };
 - ◆public: 表示is a(是一个)关系，复用基类的接口，进而实现多态
 - ◆private: 表示implemented in terms of(根据/用.....实现)的关系，复用基类的实现，而不是复用基类的接口(基类的接口被隐藏了)。不表达概念上的子类型关系，实际应用中优先考虑聚合/组合
private: 基类公有/保护成员只对当前派生类可见，对后代和外部都不可见
 - ◆protected: 基类公有/保护成员对整个继承树上的所有派生类可见，只对外部不可见。类似private，复用基类的实现，同时在多层继承中保持这种关系。在实际工程中很少使用，主要用于某些框架或库的内部实现



is a表达概念上的子类型关系

```
class Car {  
public:  
    void Run( );  
    void Brake( );  
protected:  
    int  AddWeight(int w );  
    int  ReduceWeight(int  
w );  
private:  
    int  mWeight;  
    int  mWheels;  
};
```

```
class MTCar  
    :public Car  
{  
public:  
    void FuncMT( );  
protected:  
    //...  
private:  
    //....  
};
```

```
class ATCar  
    :public Car  
{  
public:  
    void FuncAT( );  
protected:  
    //...  
private:  
    //....  
};
```



当需要一个Car时，实际给的是一个Car对象，一个MTCar对象，一个ATCar对象均可
这种语义，是水平关系无法取代的

```
class Car
{ ... };

class MTCar :public Car
{ ... };

class ATCar :public Car
{ ... };
```

```
class My {
public:
    My(Car & c)
        {pCar =&c; }
    void Func( Car & c)
        { c. Run( ); }
private:
    Car * pCar;
};
```

```
int main( ) {
    Car    car;
    MTCar  mtcar;
    ATCar  atcar;

    My mycar(atcar);
    mycar.Func(mtcar);
    ...
}
```



implemented in terms of关系表达运动员的行为实现用到了自行车的实现

```
class Bike {  
public:  
    void Move( );  
    void Stop( );  
    void Repair( );  
protected:  
    int  ChangeColor(int );  
private:  
    int  mColor;  
};
```

```
class Player : private Bike {  
public:  
    void StartRace( ) {  
        { Move( ); ... }  
    void EndRace( ) {  
        { Stop( ); ... }  
protected:  
    int  CurStrength ( );  
private:  
    int  mMaxStrength;  
    int  mAge;  
};
```



基类Pen中的公有和保护成员，在派生类Box中都是保护的，若继续以保护方式派生，那么Pen中的公有和保护成员在类Bag中，仍保持protected的访问性

```
class Pen
{
public:
    void PenPubFunc( );
protected:
    void PenProFunc( );
private:
    void PenPriFunc( );
};
```

```
class Box
:protected Pen {
public:
    void BoxPubFunc( );
protected:
    void BoxProFunc( );
private:
    void BoxPriFunc( );
};
```

```
class Bag
:protected Box {
public:
    void BagPubFunc( );
protected:
    void BagProFunc( );
private:
    void BagPriFunc( );
};
```

- ◆ 聚合/组合:
 - ◆ 具有has-a/contain-a的关系
 - ◆ 类间是水平关系，相比继承可减少类的层次，且编译期依赖性低、耦合度低
- ◆ 继承：继承方式不同，目的不同
 - ◆ public继承：表示“is-a”的关系，更强调复用接口，表达子类型关系。
 - ◆ private：表示implemented in terms of(根据/用.....实现)的关系，完全可换成聚合/组合。
 - ◆ protected：类似private，复用基类的实现，同时在多层继承中保持这种关系。
- ◆ 从复用实现的角度，优先选择聚合/组合，而不是public/private/protected继承
- ◆ 从复用接口的角度，在需要派生新的子类型的情况下，应采用public继承

```
class Bike {  
public:  
    void Move( );  
    void Stop( );  
    void Repair( );  
protected:  
    int  ChangeColor(int );  
private:  
    int  mColor;  
};
```

```
class Player {  
public:  
    void StartRace( )  
        { aBike.Move( ); //.... }  
  
    void EndRace( )  
        { aBike.Stop( ); //.... }  
protected:  
    int  CurStrength ( );  
private:  
    int  mMaxStrength;  
    int  mAge;  
    Bike aBike; // 更推荐指针形式  
};
```



本章结束